



TaskFlow

Sistema de gestión de tareas distribuido

Sistemas Distribuidos · Proyecto 2P

Docker Desktop + Kubernetes + Tailscale

Dos máquinas · dos clústeres

Máquina A: Backend + MongoDB · Máquina B: Frontend

— EL OBJETIVO

Una app de tareas partida en dos máquinas

- App de tareas: **crear, listar, editar y eliminar**.
- Backend y base de datos en la **Máquina A**. Frontend en la **Máquina B**.
- Cada máquina corre su **propio Kubernetes** de Docker Desktop. Son clústeres independientes.
- El frontend consume la API del backend por **Tailscale**, nunca por `localhost`.

— ARQUITECTURA GENERAL

Dos clústeres, un puente de Tailscale

MÁQUINA A

Backend + MongoDB

backend-service · NodePort 30080

Backend · 2 réplicas

mongodb-service · ClusterIP

MongoDB · 1 réplica + disco

TAILSCALE



IP privada : puerto

MÁQUINA B

Frontend

ConfigMap · API_URL

Frontend · 1 réplica (Nginx + React)

frontend-service · NodePort 30081

Navegador de la Máquina B

— TECNOLOGÍAS

Qué usamos y dónde

Backend (Máquina A)

- Node.js + Express
- Mongoose sobre MongoDB
- JWT + bcrypt para la autenticación

Frontend (Máquina B)

- React + Vite + Tailwind
- FullCalendar para el calendario
- Nginx sirve la app compilada

Infraestructura

- Docker Desktop · Kubernetes (kubeadm) · Tailscale · MongoDB

— DOCKER

Cada componente en su propia imagen

- Docker empaqueta código, dependencias y runtime en una **imagen** que corre igual en cualquier máquina.
- **Backend:** imagen de una etapa (Node 22 Alpine, solo dependencias de producción).
- **Frontend:** imagen de **dos etapas**. Node compila con Vite; Nginx sirve solo lo compilado, así queda liviana.
- Cada integrante prueba su imagen en local y la publica en **Docker Hub**.

`juandifrost17/taskflow-backend`

`karelgonzalez/karel-taskflow-frontend`

Los objetos que usamos

- **Deployment:** mantiene N réplicas del contenedor vivas; si un pod cae, lo recrea.
- **Service:** una dirección estable y reparto de tráfico. **ClusterIP** solo interno; **NodePort** abre un puerto hacia afuera.
- **ConfigMap:** configuración no secreta. **Secret:** contraseñas y el secreto del JWT.
- **PersistentVolumeClaim:** un disco que sobrevive al reinicio del pod (para MongoDB).

Backend y base de datos

- Backend **Deployment con 2 réplicas** (escalable a 4), con sondas de salud a `/api/health`.
- `backend-service` : **NodePort 30080**, el puerto que alcanza la Máquina B.
- MongoDB **Deployment de 1 réplica** con un **PVC** para no perder datos.
- `mongodb-service` : **ClusterIP**, la base no se expone hacia afuera.
- **Secret** (usuario y clave de Mongo, JWT) y **ConfigMap** (`MONGO_HOST` , puerto, base).

Frontend

- Frontend **Deployment de 1 réplica** (Nginx sirviendo React).
- `frontend-service` : **NodePort 30081**, por donde se abre la app en el navegador.
- **ConfigMap** con `API_URL` = IP de Tailscale de A + `30080` .
- La IP **no se quema en el código**: se inyecta al arrancar el contenedor y React la lee como `window.APP_CONFIG.API_URL` .

— TAILSCALE

La red privada que une los clústeres

- Red privada de malla (WireGuard): cada máquina de la misma tailnet recibe una **IP** `100.x.y.z` estable, sin abrir puertos en el router.
- Máquina A: `100.86.94.78` · Máquina B: `100.68.100.90` .
- El frontend usa `http://100.86.94.78:30080` como URL de la API.
- Se verifica con `tailscale status` y `tailscale ping` .

— LA CONEXIÓN

El recorrido de una petición

- 1 El navegador de B abre `localhost:30081` ; Nginx entrega la app de React.
- 2 React hace `fetch` a `100.86.94.78:30080/api/ ...` con el token JWT.
- 3 La petición viaja por Tailscale hasta el `backend-service` (NodePort), que la reparte entre las réplicas.
- 4 El backend valida el token, consulta MongoDB por su ClusterIP y responde.
- 5 La respuesta vuelve al navegador y se ven las tareas guardadas en la Máquina A.

— ESCALADO Y EVIDENCIAS

De 2 a 4 réplicas en un comando

- `kubectl scale deployment backend --replicas=4` : Kubernetes crea dos pods más y el Service reparte entre los cuatro, sin cortar el servicio.
- Evidencias de ambas máquinas: `kubectl get pods` y `get services` , `tailscale status` , `describe deployment backend` .
- Prueba de conexión cruzada, la app en el navegador con datos reales, y las imágenes publicadas en Docker Hub.

— CIERRE

Demo en vivo

El navegador de la Máquina B muestra tareas que en realidad están guardadas en la base de datos de la Máquina A. Docker las empaqueta, Kubernetes las despliega y las escala, y Tailscale conecta los dos clústeres.